# LAB 4 — VIBECODING PROMPT LOG

**Project: Breakout / Brick Breaker (Pygame)**

Name: Aaron
SRN: PES2UG24CS012

Purpose: This log records the debugging issue and feature-specific prompts used to guide the AI-assisted implementation. Each prompt states the relevant code behavior, constraints, and expected result. The code still needs to be run and verified locally; test outcomes should be filled in only after checking them in Visual Studio.

## 1. Bug fix — bricks were not removed after their hit count reached zero

**Observed issue:** The collision handler reversed the ball's vertical velocity and returned True. GameEngine.update() reduced brick.hits_remaining, but the engine did not remove the brick when the remaining hit count reached zero. As a result, normal bricks stayed on the board.

**Prompt used:**

> *Inspect game/game_engine.py, game/brick.py, and game/collision.py. Fix only the Task 1 ball–brick collision bug described in the README. Currently, handle_ball_brick_collision() detects collisions and reverses the ball's vertical velocity, while GameEngine.update() decrements brick.hits_remaining but never removes a brick when its remaining hits reach zero. Update the collision loop so that a brick is removed from self.bricks when brick.hits_remaining <= 0. Avoid modifying a list while iterating over it in an unsafe way. Preserve the existing ball bounce behavior and leave lives, game over, brick types, score, and combo multiplier unchanged. Explain the changes.*

**Expected verification:** Run python main.py, hit a normal brick, and confirm it disappears and the Bricks left counter decreases. Record the actual result after testing.

## 2. Feature — three lives, ball reset, game over, and restart

**Requirement / design reasoning:** The starter resets the ball after it falls below the paddle but does not penalize the player. The feature should track three lives, reset the combo on a miss, and stop gameplay at zero lives.

**Feature-specific prompt:**

> *Implement the lives system in GameEngine while preserving the existing controls and game loop. Start with three lives. When the ball falls below the bottom of the play area, decrement lives and reset the ball if lives remain. When lives reach zero, set a game-over state and stop normal game updates. Display the remaining lives and a clear GAME OVER message. Allow the player to press R to restart, restoring the ball, paddle, bricks, and three lives. Keep the implementation consistent with the existing Pygame architecture and explain which files changed.*

**Expected verification:** Miss the ball three times, confirm the life counter reaches zero and game over appears, then press R and verify that a new game starts. Record the actual result after testing.

## 3. Feature — normal, strong, and unbreakable brick types

**Requirement / design reasoning:** Brick behavior should be represented explicitly so that collision handling can distinguish durability from appearance. Strong bricks need multiple hits; unbreakable bricks must never be removed.

**Feature-specific prompt:**

> *Extend the existing Brick class to support normal, strong, and unbreakable types. Normal bricks must be destroyed by one hit, strong bricks must require multiple hits, and unbreakable bricks must remain on the board regardless of hits. Give each type a*

*distinct visual style and show damage progress for strong bricks where practical. Integrate this with the existing GameEngine collision loop so that only destroyed bricks are removed. Keep the default Brick constructor compatible with existing calls and avoid breaking the current renderer. Explain the design and changes.*

**Expected verification:** Hit one of each brick type and confirm normal bricks disappear after one hit, strong bricks survive the first hit and disappear after their required hits, and unbreakable bricks remain. Record the actual result after testing.

## 4. Feature — score and combo multiplier

**Requirement / design reasoning:** Scoring should reward consecutive destroyed bricks. The multiplier should increase with consecutive brick destruction and reset when the player misses the ball.

**Feature-specific prompt:**

*Add score, combo count, and a displayed combo multiplier to GameEngine. Award points only when a breakable brick is actually destroyed, not when an unbreakable brick is hit or a strong brick merely takes damage. Increase the multiplier for consecutive brick destructions, with a reasonable cap so the score remains understandable. Reset the combo and multiplier when the ball falls below the paddle. Display score and multiplier in the HUD, and keep these values consistent when restarting the game. Integrate with the brick types and collision behavior without duplicating collision logic. Explain the scoring rules and changed files.*

**Expected verification:** Destroy consecutive breakable bricks and confirm score and multiplier increase. Miss the ball and confirm the multiplier resets. Verify that hits on unbreakable bricks do not award destruction points. Record the actual result after testing.

## 5. Integration / regression check — ensure all features work together

**Purpose:** The features share collision, game-state, rendering, and restart logic, so the integrated version should be checked for regressions before recording the after-video.

**Integration prompt:**

*Review the integrated Breakout implementation against every requirement in README.md. Check for incompatible method names or assumptions across Brick, collision handling, GameEngine, and renderer. Check that ball collisions do not repeatedly register while overlapping a brick, that destroyed bricks are safely removed, unbreakable bricks never disappear, misses reset the combo, game over stops updates, and pressing R fully resets the game. Make only necessary corrections, preserve the project's structure, and provide a concise list of manual tests to run. Do not claim a test passed unless it was actually run.*

**Expected verification:** Run the game locally and complete the feature checklist. Only after successful verification, record the 10-second after-video and push the updated code to the personal fork.

## Testing status

The prompts and expected checks are documented here. Fill in test outcomes only after running the updated project locally. Do not mark a feature as passed based only on code review.

## Submission reminders

- Keep the original 10-second before-video.
- Record a 10-second after-video only after all features work.
- Push the updated code to the personal fork; do not open a pull request to SETAPESU26.
- Submit the videos, updated code, and required chat/LLM history under the Lab-4 folder as instructed.